

Nutanix Tenant FinOps & Monitoring

Infrastructure · Performance · Security · Throughput · AI Token Billing

Note on pricing data: Nutanix on-prem does not publish per-resource public pricing. All unit prices below are **illustrative estimates** derived from typical private cloud TCO models and Nutanix NCM Cost Governance conventions. Replace with actual hardware amortisation + software licence costs from your specific deployment before using for chargebacks.^[1]

Architecture Overview

```
Nutanix Cluster (AHV / Prism Central)
├─ Prism Central v4 API (:9440) ← primary metrics source
├─ Nutanix Objects Prometheus Exporter (:9440/oss/api/nutanix/metrics)
├─ Nutanix Enterprise AI (NAI 2.5+) ← AI token metrics
│   ├── LLM Inference endpoints (OpenAI-compatible)
│   ├── AI Agent Gateway (NAI 2.7)
│   └─ NAI Metrics API + OTEL export
└─ IPMI / Redfish (physical node metrics)
    ⇕
nutanix-prometheus-exporter (Python, port :8000)
    ⇕
Prometheus → Grafana
```

Data collection uses the community **nutanix-prometheus-exporter** in v4 mode, which targets Prism Central and multithreads v4 API calls for performance. NAI metrics are collected via the NAI Management API and optionally via the OpenTelemetry Collector export configured in NAI 2.5+.^{[2][3]}

Part 1 — Infrastructure Performance Metrics (Prism Central v4 API)

1.1 Cluster-Level Metrics

Collected via GET `/api/nutanix/v4.0/vmm/v4/stats/clusters/{clusterExtId}/stats` or the legacy `v2.0/cluster` endpoint.^{[4][5]}

Metric	API Field	Unit	FinOps / Ops Value
Cluster CPU utilisation	<code>hypervisor_cpu_usage_ppm ÷ 10000</code>	%	Capacity headroom; over-committed clusters inflate VM costs
Cluster memory utilisation	<code>hypervisor_memory_usage_ppm ÷ 10000</code>	%	Memory pressure drives ballooning and swap costs
Cluster storage utilisation	<code>storage_usage_bytes / storage_capacity_bytes</code>	%	Storage tier fill rate; triggers tiering cost
Cluster IOPS (read + write)	<code>controller_num_read_io + controller_num_write_io</code>	IOPS	I/O hotspot detection
Cluster I/O latency	<code>controller_avg_io_latency_usecs</code>	µs	SLA compliance; high latency = workload impact
Cluster throughput	<code>controller_io_bandwidth_kBps</code>	KB/s	Network + storage throughput per cluster
Number of VMs running	Prism v4 vmm list + filter by <code>power_state=ON</code>	count	Denominator for per-VM cost allocation
Number of VMs off	Prism v4 vmm list + filter <code>power_state=OFF</code>	count	Idle resource detection — FinOps waste signal
Node count	GET <code>/api/nutanix/v4.0/clustermgmt/v4/config/clusters/{id}/hosts</code>	count	Licence cost base

1.2 Per-VM Metrics

Collected via GET `/api/nutanix/v4.0/vmm/v4/stats/ahv/vms/{vmExtId}/stats.`^{[6][5]}

Metric	API Field	Unit	FinOps / Ops Value
vCPU usage	hypervisor_cpu_usage_ppm	%	Per-VM chargeback basis
Memory usage	guest.memory_usage_bytes	bytes	Per-VM chargeback basis
Disk I/O read	controller_num_read_io	IOPS	Storage tier attribution
Disk I/O write	controller_num_write_io	IOPS	Storage tier attribution
Disk read latency	controller_avg_read_io_latency_usecs	µs	Performance SLA per VM
Disk write latency	controller_avg_write_io_latency_usecs	µs	Performance SLA per VM
NIC received bytes	network.received_bytes	bytes	Network I/O cost attribution
NIC transmitted bytes	network.transmitted_bytes	bytes	Network I/O cost attribution
VM power state	power_state	ON/OFF/SUSPEND	Idle/zombie VM detection
VM uptime hours (derived)	Calculated from power_on_timestamp	hours	Time-based billing

Key alert: VMs in OFF state for >14 days with allocated storage are FinOps waste — storage cost continues to accrue for powered-off VMs.[^1]

1.3 Storage Container / Volume Metrics

Collected via GET /api/nutanix/v4.0/volumes/v4/... and the Objects Prometheus Exporter.[^7]

Metric	Source	Unit	FinOps / Ops Value
Container used capacity	Prism v4 volumes API	bytes	Storage cost per project/team
Container provisioned capacity	Prism v4 volumes API	bytes	Thin provisioning waste
Deduplication savings ratio	data_reduction_ratio	ratio	Effective cost per raw GB
Compression ratio	compression_saving_ratio	ratio	Reduces storage cost per VM
Snapshot storage used	snapshot.used_bytes	bytes	Snapshot retention cost
Object store buckets	Objects exporter nutanix_objects_bucket_count	count	Tenant bucket inventory

Metric	Source	Unit	FinOps / Ops Value
Object store capacity used	Objects exporter nutanix_objects_used_bytes	bytes	Per-bucket chargeback
Object store GET/s	Objects exporter nutanix_objects_get_per_second	req/s	Throughput SLA
Object store PUT/s	Objects exporter nutanix_objects_put_per_second	req/s	Throughput SLA
Object store throughput in	Objects exporter nutanix_objects_rx_throughput_kbps	KB/s	Bandwidth cost
Object store throughput out	Objects exporter nutanix_objects_tx_throughput_kbps	KB/s	Bandwidth cost

1.4 Physical Node / Hardware Metrics

Collected via the Redfish/IPMI mode of the nutanix-prometheus-exporter.^[^2]

Metric	Unit	FinOps / Ops Value
Node power consumption	Watts	Electricity cost attribution per node / rack
Node CPU temperature	°C	Thermal throttling risk; cooling cost driver
Node memory temperature	°C	Thermal health
Fan speed	RPM	Cooling system health
Node disk health (SMART)	status	Predictive failure → prevent data loss cost

Part 2 — FinOps Chargeback Model

The following model maps raw resource metrics to cost. All prices are **illustrative estimates** — replace with actual values.^{[8][1]}

2.1 Unit Price Reference (Illustrative)

Resource	Unit	Illustrative Price	Basis
vCPU-hour (active)	vCPU × hour	SGD 0.02/vCPU-hr	Amortised server + licence cost
vCPU-hour (idle/off VM)	vCPU × hour	SGD 0.005/vCPU-hr	Reduced rate for reserved capacity
RAM-hour	GB × hour	SGD 0.008/GB-hr	Amortised memory cost
Storage (SSD tier)	GB × hour	SGD 0.00015/GB-hr	NVMe SSD amortisation
Storage (HDD tier)	GB × hour	SGD 0.00004/GB-hr	HDD amortisation
Snapshot storage	GB × hour	SGD 0.00010/GB-hr	Snapshot retention cost
Object storage	GB × month	SGD 0.012/GB-mo	Nutanix Objects overhead
Object GET request	per 10,000	SGD 0.004	S3-equivalent egress ops

Resource	Unit	Illustrative Price	Basis
Object PUT request	per 10,000	SGD 0.005	S3-equivalent ingress ops
Network egress (inter-cluster)	GB	SGD 0.02/GB	Network infrastructure cost
GPU node (H100, full node)	node-hour	SGD 8.50/hr	GPU server amortisation + power
GPU node (A100, full node)	node-hour	SGD 5.20/hr	GPU server amortisation + power
GPU MIG slice (1/7 H100)	slice-hour	SGD 1.30/hr	MIG fractional GPU

2.2 Chargeback Formula per VM (hourly)

```

VM_cost_per_hour =
  (vCPU_allocated × price_vcpu_hr)
+ (RAM_GB_allocated × price_ram_hr)
+ (disk_GB_provisioned × price_storage_tier_hr)
+ (snapshot_GB × price_snapshot_hr)
+ (network_egress_GB × price_network_GB)

```

For powered-off VMs: charge only storage + snapshot (no CPU/RAM).[¹]

2.3 Prometheus Recording Rules for FinOps

```

# In Prometheus rules file: nutanix_finops.rules.yaml
groups:
- name: nutanix_finops
  rules:

  # Per-VM hourly cost (SGD)
  - record: vm:cost_per_hour:sgd
    expr: |
      (nutanix_vm_vcpu_allocated * 0.02)
      + (nutanix_vm_ram_gb_allocated * 0.008)
      + (nutanix_vm_storage_ssd_gb * 0.00015 * 24)
      + (nutanix_vm_storage_hdd_gb * 0.00004 * 24)

  # Monthly cost projection per VM
  - record: vm:cost_per_month_projected:sgd
    expr: vm:cost_per_hour:sgd * 24 * 30

  # Waste: powered-off VMs with allocated storage > 50GB
  - record: vm:waste_candidates
    expr: |
      nutanix_vm_power_state == 0
      and nutanix_vm_storage_ssd_gb > 50

  # Cluster efficiency: vCPU used vs provisioned
  - record: cluster:vcpu_efficiency_ratio
    expr: |
      sum(nutanix_vm_vcpu_usage_pct * nutanix_vm_vcpu_allocated) by (cluster)
      / sum(nutanix_vm_vcpu_allocated) by (cluster)

```

2.4 FinOps Dashboard Panels

Panel	Metric(s)	Chart Type
Monthly cost by VM (top 20)	<code>vm:cost_per_month_projected:sgd</code>	Bar chart, sorted desc
Monthly cost by project/category tag	Group <code>vm:cost_per_month_projected:sgd</code> by category label	Pie / treemap
Powered-off VM waste (SGD/month)	<code>vm:waste_candidates</code> × storage cost	Stat panel — alert if > SGD 500
vCPU efficiency ratio	<code>cluster:vcpu_efficiency_ratio</code>	Gauge — target > 60%
Storage utilisation trend	<code>nutanix_storage_usage_pct</code>	Time series
Dedup + compression savings (SGD equivalent)	$(1 - 1/\text{dedup_ratio}) \times \text{storage_cost_raw}$	Stat panel
Object storage cost by bucket	Objects exporter + price	Bar chart
Snapshot cost by VM	<code>nutanix_vm_snapshot_gb</code> × 0.0001	Table
Cost forecast (30-day)	<code>vm:cost_per_hour:sgd</code> × 24 × 30 rolling sum	Time series + forecast

Part 3 — Nutanix Enterprise AI (NAI) Token Billing

NAI 2.5+ exposes a Metrics API and supports OpenTelemetry export for LLM endpoint observability.^{[9][10][^3]}

3.1 NAI Metrics API Endpoints

```
# Token usage per endpoint
GET /api/enterpriseai/v1/metrics/endpoints/{endpointId}/stats
# Filter by time range and metric type

# Usage by API key (for per-team billing)
GET /api/enterpriseai/v1/metrics/apikeys/{apikeyId}/stats

# Cluster-wide aggregated metrics
GET /api/enterpriseai/v1/metrics/cluster/stats
```

3.2 Available NAI LLM Metrics^{[11][9]}

Metric Name	Type	Unit	Description
<code>inputTokenCount</code>	Counter	tokens	Total prompt/input tokens processed
<code>outputTokenCount</code>	Counter	tokens	Total completion/output tokens generated
<code>llmMetricsApiUsageCount</code>	Counter	requests	Total API requests

Metric Name	Type	Unit	Description
timeToFirstTokenAverage	Gauge	ms	TTFT average — user-facing latency
timeToFirstTokenPercentile99	Gauge	ms	TTFT P99 — SLA compliance
timeToFirstTokenPercentile95	Gauge	ms	TTFT P95
timeToFirstTokenPercentile50	Gauge	ms	TTFT median
timePerOutputTokenAverage	Gauge	ms/token	TPOT average — throughput indicator
timePerOutputTokenPercentile99	Gauge	ms/token	TPOT P99
timePerOutputTokenPercentile95	Gauge	ms/token	TPOT P95
outputTokensPerSecond	Gauge	tokens/s	Generation throughput
activeRequests	Gauge	count	Concurrent in-flight requests
queuedRequests	Gauge	count	Requests waiting for GPU capacity
cachedTokenUsage	Counter	tokens	vLLM KV-cache hits — cost efficiency signal
requestSuccessCount	Counter	requests	Successful completions
requestFailureCount	Counter	requests	Failed requests (by error type)
gpuUtilisation	Gauge	%	GPU compute utilisation per endpoint
gpuMemoryUtilisation	Gauge	%	GPU VRAM utilisation

3.3 Token Billing Model (Illustrative Pricing)

NAI runs models on your own Nutanix hardware, so the billing is an **internal chargeback** of compute cost, not external API billing. The GPU node cost is amortised to per-token rates.

Model Class	Input Token Rate	Output Token Rate	Basis
Small ($\leq 7B$, e.g. Llama-3.1-8B)	SGD 0.00000050/token	SGD 0.00000150/token	1× H100 MIG slice, amortised
Medium (8–30B, e.g. Llama-3.3-70B 4-bit)	SGD 0.00000120/token	SGD 0.00000400/token	2× H100 GPU-passthrough
Large (31–72B, e.g. Llama-3.3-70B FP16)	SGD 0.00000280/token	SGD 0.00000900/token	4× H100 GPU-passthrough
Embedding (e.g. BGE-M3)	SGD 0.00000010/token	N/A	0.5× H100 MIG slice
Reranker	SGD 0.00000020/token	N/A	0.25× H100 MIG slice
Image generation	SGD 0.0002/image	N/A	Per generated image

Derivation method: $\text{GPU_node_cost_per_hour} \div (\text{tokens_per_second} \times 3600)$ gives approximate cost per token at typical utilisation.^[1]

3.4 Per-API-Key Token Accumulator (Prometheus)

The NAI Management API provides per-API-key usage attribution. Build a Prometheus Pushgateway-based collector:¹²

```
import requests
from prometheus_client import CollectorRegistry, Gauge, Counter, push_to_gateway

NAI_BASE = "https://nai.example.com/api/enterpriseai/v1"
HEADERS = {"Authorization": "Basic <token>"}

# Fetch per-API-key metrics
apikeys = requests.get(f"{NAI_BASE}/apikeys", headers=HEADERS).json()

registry = CollectorRegistry()
input_tokens = Counter('nai_input_tokens_total', 'Input tokens', ['api_key_name',
output_tokens = Counter('nai_output_tokens_total', 'Output tokens', ['api_key_name',
cost_sgd = Gauge('nai_cost_sgd', 'Cost SGD', ['api_key_name',

for key in apikeys['data']['apikeys']:
    stats = requests.get(f"{NAI_BASE}/metrics/apikeys/{key['id']}/stats", headers=HE
    inp = stats['data']['inputTokenCount']
    out = stats['data']['outputTokenCount']
    model_class = classify_model(key['endpoints']['name']) # maps to small/medium/l
    cost = (inp * INPUT_PRICE[model_class]) + (out * OUTPUT_PRICE[model_class])

    input_tokens.labels(key['name'], key['endpoints']['name'], model_class).inc(inp)
    output_tokens.labels(key['name'], key['endpoints']['name'], model_class).inc(out
    cost_sgd.labels(key['name'], key['endpoints']['name'], model_class).set(cost)

push_to_gateway('prometheus-pushgateway:9091', job='nai_billing', registry=registry)
```

3.5 AI Token Billing Dashboard Panels

Panel	Metric	Chart Type
Total input tokens today / this month	sum(nai_input_tokens_total)	Stat + sparkline
Total output tokens today / this month	sum(nai_output_tokens_total)	Stat + sparkline
Token cost by API key (top 10)	nai_cost_sgd grouped by api_key_name	Bar chart, sorted desc
Token cost by model	nai_cost_sgd grouped by model	Pie chart
Input vs output token ratio	sum(nai_input_tokens_total) / sum(nai_output_tokens_total)	Gauge — high ratio = large prompts
Cache hit rate (vLLM KV)	nai_cached_tokens / nai_input_tokens_total	% gauge — >30% is efficient
TTFT P99 by endpoint	nai_ttft_p99_ms	Time series — SLA line at 3000ms

Panel	Metric	Chart Type
TPOT P95 by model	<code>nai_tpot_p95_ms_per_token</code>	Time series
Tokens/second throughput	<code>nai_output_tokens_per_second</code>	Time series by endpoint
GPU utilisation vs token throughput	<code>nai_gpu_utilisation_pct</code> vs <code>nai_output_tokens_per_second</code>	Dual-axis time series
Queued requests	<code>nai_queued_requests</code>	Gauge — alert if > 10 for > 60s
Monthly projected AI cost (SGD)	<code>sum(nai_cost_sgd) * (30 * 24 / hours_elapsed)</code>	Stat panel

Part 4 — Security Monitoring Statistics

4.1 Prism-Level Security Metrics

Metric	Source	Alert Condition
Failed Prism Central logins	Prism audit log → log parse	>5 failures from same IP in 5 min
Admin API calls outside business hours	Prism v4 audit API	Any write/delete at 22:00–06:00
VM disk snapshot creation rate	Prism v4 dataprotection API	>10 snapshots in 5 min = ransomware signal
VM cloning rate	Prism v4 vmm clone events	Spike = lateral movement indicator
Firewall rule changes (Flow Microsegmentation)	Prism v4 microseg API	Any change outside change window
Anomalous cross-cluster data transfer	Objects throughput spike	> 3σ above baseline
Root/admin role assignment	Prism IAM v4 audit	Any occurrence → immediate alert

4.2 NAI Security Metrics

Metric	Source	Alert Condition
Jailbreak detection rate	NAI response <code>content_filter_results.jailbreak.detected</code>	Any <code>true</code> → log to SIEM
Hate/violence content filtered	NAI response <code>content_filter_results.hate.filtered</code>	Any <code>true</code> → log with <code>api_key_name</code>
API key requests exceeding rate limit	NAI 429 response rate per <code>api_key</code>	>0 sustained → potential token farming
Requests to deleted/disabled endpoint	NAI 404 on endpoint	May indicate probing
Abnormally large prompt tokens	<code>inputTokenCount</code> per request > <code>context_length</code> × 0.9	Potential prompt injection attempt

Metric	Source	Alert Condition
New API key created outside hours	NAI audit log eventType=Create on entityType=APIKey	Alert if outside business hours

Part 5 — Prometheus Scrape Configuration for Nutanix

```
scrape_configs:

  # Nutanix Prism Central (via nutanix-prometheus-exporter in v4 mode)
  - job_name: nutanix_prism
    static_configs:
      - targets: ['nutanix-exporter:8000']
    scrape_interval: 30s      # v4 API rate limits – don't go below 15s

  # Nutanix Objects Storage Prometheus Exporter (native)
  - job_name: nutanix_objects
    metrics_path: /oss/api/nutanix/metrics
    scheme: https
    tls_config:
      insecure_skip_verify: false
    basic_auth:
      username: <pc-username>
      password: <password>
    static_configs:
      - targets: ['prism-central:9440']
    scrape_interval: 60s

  # NAI Token Billing (via Pushgateway – polled by billing collector script)
  - job_name: nai_billing
    static_configs:
      - targets: ['prometheus-pushgateway:9091']

  # NAI OpenTelemetry export (if NAI OTEL Collector configured)
  - job_name: nai_otel
    static_configs:
      - targets: ['nai-otel-collector:8889']
    scrape_interval: 15s
```

Part 6 — Unified Grafana Dashboard Hierarchy

Dashboard	Sub-Panels
Cluster Overview	CPU %, RAM %, Storage %, IOPS, latency, VM count on/off, node count
FinOps: VM Chargeback	Cost/VM/month, cost by project tag, waste VMs, vCPU efficiency, storage tier breakdown
FinOps: Storage Costs	Container usage, snapshot costs, dedup savings, object store cost by bucket
AI Token Billing	Tokens by API key, tokens by model, cost SGD today/month, cache hit rate, projected monthly AI spend

Dashboard	Sub-Panels
AI Performance	TTFT P50/P95/P99, TPOT P95/P99, tokens/sec, GPU utilisation, queued requests
Security Signals	Prism failed logins, off-hours admin ops, snapshot creation rate spike, NAI jailbreak events, content filter events
Physical Health	Node power (Watts), temperatures, fan RPM, disk SMART status

Part 7 — Instrumentation Checklist

Component	Action Required
Prism Central	Enable REST API access; create read-only monitoring service account; deploy nutanix-prometheus-exporter in v4 mode ^[^2]
Nutanix Objects	Enable Prometheus exporter (available from Objects 3.5.1) ^[^7]
NAI 2.5+	Configure OpenTelemetry Collector export via PATCH <code>/api/enterpriseai/v1/cluster/config</code> with OTEL endpoint ^[^3] ; enable rsyslog to forward audit logs to SIEM
NAI API keys	One API key per team/project — never shared keys — to enable per-team billing attribution ^[^12]
IPMI/Redfish	Enable Redfish on physical nodes; configure redfish mode in nutanix-prometheus-exporter ^[^2]
NCM Cost Governance	Configure TCO buckets (Hardware, Software, Facilities, Telecom, Services, People) per cluster — provides native per-VM cost view in Prism UI ^[^1]

References

1. [Total Cost of Ownership for Nutanix Infrastructure | Nutanix ...](#) - See how NCM Cost Governance displays a true TCO for your Nutanix clusters by showing hardware/software...
2. [sbourdeaud/nutanix-prometheus-exporter](#) - Contribute to sbourdeaud/nutanix-prometheus-exporter development by creating an account on GitHub.
3. [Nutanix Enterprise AI 2.5 - Configuring OpenTelemetry ...](#) - Configure OpenTelemetry Collector to export the metrics available in Nutanix Enterprise AI and view ...
4. [API Reference v4 Introduction - Nutanix.dev](#)
5. [Using the Nutanix v4 API Python SDK to extract performance metrics](#)
6. [stats_api — VMM-SDK documentation](#)
7. [Monitoring Nutanix Objects Storage with Prometheus/Grafana](#)
8. [Streamline Hybrid Cloud Metering and Charging with Exivity](#)
9. [Nutanix Enterprise AI Performance and Observability](#) - Usage metrics: Tracks the total API requests, successful and failed requests, token usage (input and...
10. [Nutanix Turns AI Ambition into Enterprise Control and Customer ...](#) - Nutanix Enterprise AI (NAI) solution is built to provide a simple and intuitive infrastructure platf...

11. Nutanix Enterprise AI 2.4 - A powerful step forward in performance, control, and simplicity. Nutanix Enterprise AI 2.4 introduces...
12. Nutanix Enterprise AI (2.5.0) - API Documentation